

Recursive List Processing

Learning Outcomes

by the **End of the Lecture, Students** that **Complete** the **Recommended Exercises** should be **Able** to:

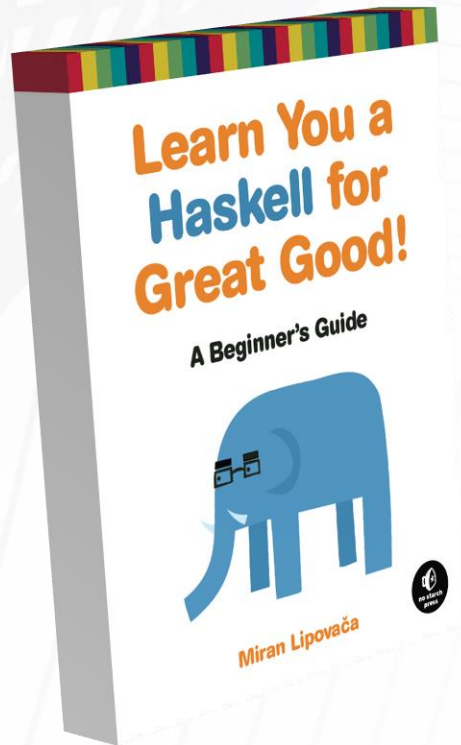
Practice Primitive and General Recursion

Use the Accumulation Design Pattern

Use the Transformation Design Pattern

Use the Selection Design Pattern

Reading and Suggested Exercises



Read the following Chapter(s):

Chapter 5:
"A few more recursive functions"

Primitive Recursion with Lists

as noted previously, **Primitive Recursion** is
Defined with a **Base Case** at $n = 0$ and an
Inductive Case at $n > 0$ such that the **Argument**
for Each **Recursive Call** is $n - 1$

there is, however, a way that the **Primitive Recursion**
Pattern can be **Applied** to **Lists**

Primitive Recursion with Lists

every **List** is **either Empty** (for which `[]` is the **Constructor**) or an **Applied Constructor** `(h : t)` (for which `:` is the **Constructor** and `t` is a **List**)

the **Empty List** has a **Length** of **Zero** and **Any Non-Empty List** of **Length** n combines the **Head Element** with a **List** of **Length** $n - 1$

consequently, **Recursive List Manipulating Functions** frequently follow a certain **Form...**

Primitive Recursion with Lists

...a **Recursive List Manipulating Function**
frequently takes the **Form**

`foo [] = something`

`foo (head : tail) = something foo (tail) something`

n.b., do not ignore the parentheses unless you actually want `(foo head) : tail`

Recursion is then **Applied** to the **List Argument** `tail`
(thereby **Decreasing** the **Problem Size**)

Primitive Recursion with Lists

How would you write a **Recursive Function** to...

Compute the Length of a **List of Integers**

?

```
foo [] = something
```

```
foo (head : tail) = something foo (tail) something
```

Primitive Recursion with Lists

```
len :: [Int] -> Int
```

```
len [] = 0
```

```
len ( _ : tail ) = 1 + len tail
```


Primitive Recursion with Lists

How would you write a **Recursive Function** to...
Find the Maximum Element of a **List of Integers**

?

```
foo [] = something
```

```
foo (head : tail) = something foo (tail) something
```

Primitive Recursion with Lists

How would you write a **Recursive Function** to...

Test if a specific **Element** belongs to a **List**

?

```
foo [] = something
```

```
foo (head : tail) = something foo (tail) something
```

Fundamental Recursive Patterns

there are **Three Fundamental Patterns** for **Recursion on Lists** that can be **Widely Applied**

Accumulation (i.e., "Folding")

Apply a binary **Operator Between** the **Elements** of a **List**

Transformation (i.e., "Mapping")

Call a **Function** on **Every Element** in a **List**

Selection (i.e., "Filtering")

Test a **Predicate** for **Every Element** of a **List**

Recursive Accumulation Pattern

a **Function** that is applying the **Accumulation** (i.e., "**Folding**") **Pattern** to a **Binary Operator** *oper* would take a **List Argument** $[a_1, a_2, a_3, \dots, a_n]$ and **Produce** $a_1 \text{ oper } a_2 \text{ oper } a_3 \dots \text{ oper } a_n$ as a **Result**

How would you handle the **Recursive Case** to "**Fold**"?

How would you handle the **Base Case**?

Recursive Accumulation Pattern

How would you handle the **Recursive Case**?

```
foldOper (head : tail) = something foldOper(tail) something
```

Where do you **Apply** the **Operator**?

What do you **Do** with the **Head**?

Recursive Accumulation Pattern

the **Binary Operator** requires **Two Operands**

the **Head** can be the **Left**

since `foldOper anyList` should **Return** a **Single Result**
(i.e. the **Result** when `oper` is **Folded** into `anyList`) the
Return Value of the **Recursive Call** can be the **Right**

$$\text{foldOper } (h:t) = h \text{ oper foldOper}(t)$$

Recursive Accumulation Pattern

How would you Handle the Base Case?

`foldOper [] = something`

there are **Not Enough Operands** for **Any Applications** of the **Operator**, so what is the **Only Possible Return** ?

Recursive Accumulation Pattern

the **Identity Element** of an **Operator** is a Value which, when **Used** as an **Operand** in a **Binary Operation**, leaves the **Other Operand Unaffected**

as a **Clarifying Example**, consider the **Effect** of a **Zero** on a **Running Total** (or a **One** on a **Running Product**)

`foldOper [] = identity element of oper`

Recursive Accumulation Pattern

Computing the Sum of the Elements of a List of Integers is an Example of the Accumulation Pattern

it is the **Folding** of the **Addition Operator**

How would you Write a Recursive Function that Sums All the Elements of a List of Integers?

Recursive Accumulation Pattern

```
sum :: [Int] -> Int
sum [] = 0
sum (h:t) = h + sum t
```

How would you **Trace** the following **Evaluation**?

```
sum [1, 2, 4, 8, 16]
```


Recursive Accumulation Pattern

```
sum [1, 2, 4, 8, 16]
~ sum 1 : [2, 4, 8, 16]
~ 1 + sum [2, 4, 8, 16]
~ 1 + sum 2 : [4, 8, 16]
~ 1 + 2 + sum [4, 8, 16]
~ 1 + 2 + sum 4 : [8, 16]
~ 1 + 2 + 4 + sum [8, 16]
~ 1 + 2 + 4 + sum 8 : [16]
~ 1 + 2 + 4 + 8 + sum [16]
~ 1 + 2 + 4 + 8 + sum 16 : []
~ 1 + 2 + 4 + 8 + 16 + sum []
~ 1 + 2 + 4 + 8 + 16 + 0
```

Recursive Transformation Pattern

a **Function** that is applying the **Transformation** (i.e., "**Mapping**") **Pattern** to a **Function** *func* would take a **List Argument** $[a_1, a_2, a_3, \dots, a_n]$ and **Produce** $[(func\ a_1), (func\ a_2), \dots, (func\ a_n)]$

How would you handle the **Recursive Case** to "**Map**"?

How would you handle the **Base Case**?

Recursive Transformation Pattern

How would you handle the **Recursive Case**?

```
mapFunc (h:t) = something mapFunc(t) something
```

What is the **Argument** to the **Function**?

What do you **Do** with the **Tail**?

Recursive Transformation Pattern

How would you Handle the Base Case?

```
mapFunc [] = something
```

there is **No Possible Argument** for the **Function**,
so what is the **Only Possible Return** ?

Recursive Selection Pattern

a **Function** that is applying the **Selection** (i.e., "**Filtering**") **Pattern** with a **Predicate** *pred* would take a **List Argument** $[a_1, a_2, a_3, \dots, a_n]$ and **Produce** a **List** of those **Elements** that **Satisfy** *pred*

How would you handle the **Recursive Case** to "**Filter**"?

How would you handle the **Base Case**?

Recursive Selection Pattern

How would you handle the **Recursive Case**?

`filterPred (h:t) = something filterPred(t) something`

What is the **Difference** in the **Result** when **Head Does Satisfy** the **Predicate** vs. when it **Does Not**?

Additional Recursive Design Exercises

How would you **Write Recursive Functions** to...

Double Each Element of a List

`doubleEach :: [Int] -> [Int]`
`doubleEach [2, 4] == [2, 4, 8]`
`doubleEach [Int] -> [Int]`

Return only the Even Elements of a List

`filterEven :: [Int] -> [Int]`
`filterEven [1, 2, 3, 4, 5, 6] == [2, 4, 6]`

Concatenate any number of Lists

Additional Recursive Design Exercises

How would you **Write Recursive Functions** to...

Double Each Element of a List

```
doubleAll [1, 2, 4] = [2, 4, 8]  
doubleAll :: [Int] -> [Int]
```

Return only the Even Elements of a List

```
selectEven [1, 2, 4] = [2, 4]  
selectEven :: [Int] -> [Int]
```

Concatenate any number of Lists

```
concatLists [[1, 2], [3], [4, 5, 6]] = [1, 2, 3, 4, 5, 6]  
concatLists :: [[a]] -> [a]
```

Additional Recursive Design Exercises

How would you **Write Recursive Functions** to...

Remove a Specific Element from a List

Create a String of n Asterisks...

...then **Use it to Make a Histogram Creator**

Additional Recursive Design Exercises

How would you **Write Recursive Functions** to...

Remove a Specific Element from a List

```
remove :: (Eq a) => a -> [a] -> [a]  
remove 3 [1, 2, 3, 4, 3, 1] = [1, 2, 4, 1]
```

Create a String of n Asterisks...

```
bar 3 = "***"  
bar :: Int -> [Char]
```

...then **Use it to Make a Histogram Creator**

```
histogram [3, 4, 1] = ["***", "****", "*"]  
histogram :: [Int] -> [[Char]]
```